PYTHON SUMMER PLAN · DAY 18

Build day: the autoclicker

Wire decorators (Day 15) + global hotkeys (Day 17) into one working tool.

Thu May 28, 2026 · ~75 min · Yuvraj Ruparel

RECAP · WHAT WE'RE COMBINING

Two halves snap together today

Day 15 — decorators

A decorator wraps a function to add behaviour without touching its body. Today we use that shape to register a hotkey: put @hotkey above a function and it fires on a key combo.

Day 17 — pyautogui + hotkeys

pyautogui.click() moves/clicks the real mouse. A global hotkey listens system-wide, even when your terminal isn't focused. Combine them: a key toggles an automatic clicking loop.

01 · HOW THE PIECES FIT

The shape of an autoclicker

What this page is: the mental model before any code — three parts running at once.

1. A state flag

One boolean, clicking. True = spam clicks, False = idle. Everything reads this flag.

2. A click loop on its own thread

A background thread loops: if clicking, click then sleep(1/cps). It must be separate so the hotkey stays responsive.

3. A global hotkey to flip the flag

Press the combo → flag flips. The loop notices on its next pass and starts/stops.

02 · WHY A SEPARATE THREAD

Don't block the listener

What this page is: clicking in a plain loop freezes everything. A thread fixes it.

```
import threading, time, pyautogui

clicking = False
cps = 10            # clicks per second

def click_loop():
    while True:
        if clicking:
            pyautogui.click()
        time.sleep(1/cps)

# start the loop in the background, daemon=dies with program
t = threading.Thread(target=click_loop, daemon=True)
t.start()
```

› The loop runs forever

It clicks only while the flag is True, otherwise just idles and sleeps.

› daemon=True

The thread shuts down automatically when the main program exits. No hang on quit.

› Listener stays free

Because clicking happens on this thread, the main thread is free to watch for your hotkey.

03 · CLICK RATE

Controlling clicks per second

What this page is: how sleep(1/cps) sets the speed, and how to change it live.

```
cps = 10

# bigger cps -> shorter sleep -> faster clicking
time.sleep(1 / cps)   # 10 cps -> 0.1s gap

def set_cps(new):
    global cps
    cps = max(1, min(50, new))
    print(f"cps = {cps}")
```

OUTPUT

```
cps = 25
cps = 50   # clamped at ceiling
```

› The math

Gap between clicks = 1/cps seconds. 10 cps = one click every 0.1 s.

› Clamp it

max(1, min(50, new)) keeps cps in a sane 1–50 range. Stops a typo setting 9999.

› global keyword

set_cps rebinds the module-level cps, so the running loop picks up the new value.

@hotkey registers the toggle

What this page is: the Day-15 decorator shape used to bind a function to a key combo.

```
import keyboard

# decorator: bind any function to a key combo
def hotkey(combo):
    def wrap(fn):
        keyboard.add_hotkey(combo, fn)
        return fn
    return wrap

@hotkey("ctrl+shift+1")
def toggle():
    global clicking
    clicking = not clicking
```

› Decorator = wrapper

hotkey(combo) returns wrap, which registers fn then hands it back unchanged.

› Reads top to bottom

@hotkey("ctrl+shift+1") above toggle = "run this on that combo."

› Toggle flips the flag

One press starts the clicker, next press stops it. The loop reacts on its own.

05 · MINECRAFT MODE

Hold-to-spam clicking

What this page is: instead of a toggle, click only while a key is physically held down.

```
# toggle mode: press once on, press again off
# hold mode:  click ONLY while key is down

def hold_loop(key):
    while True:
        if keyboard.is_pressed(key):
            pyautogui.click()
        time.sleep(1/cps)

# run it watching the 'r' key
threading.Thread(target=hold_loop,
    args=("r",), daemon=True).start()
```

› is_pressed(key)

Returns True for as long as the key is held. The loop clicks each pass while held.

› Why "minecraft mode"

Mirrors how players hold a button to auto-attack — release to stop instantly.

› Pick one mode

Run either the toggle or the hold loop, not both on the same flag, or they fight.

06 · THE WHOLE TOOL AT A GLANCE

Four parts, one file

Part	What it is	Key call	Why it matters
State flag	module-level bool	clicking = not clicking	single source of truth
Click loop	background thread	pyautogui.click()	clicks without freezing UI
Rate control	clamped int	time.sleep(1/cps)	tune speed safely 1–50
Hotkey	decorator binding	keyboard.add_hotkey	start/stop system-wide

💡 **Takeaway**

All four parts share one global flag. The hotkey writes it, the loop reads it — that's the entire coordination mechanism.

TODAY'S EXERCISE · 75 MIN

autoclicker.py

Build one script that auto-clicks the mouse, toggled by a global hotkey, with an adjustable click rate and a separate hold-to-spam "minecraft" mode.

STRUCTURE

- click_loop() on a daemon thread
- global clicking flag + cps
- clean Ctrl+C / esc to quit

HOTKEYS

- ctrl+shift+1 → toggle on/off
- ctrl+shift+↑/↓ → cps up/down
- register each via @hotkey decorator

BEHAVIOR

- toggle mode: press on, press off
- minecraft mode: hold key to spam
- print state changes to terminal

TIPS & GOTCHAS

Things that will bite you

⚠ keyboard lib needs sudo on macOS

The keyboard library often needs sudo / root on macOS and is flaky there. I'm not certain it works cleanly on your setup — pynput is the more reliable mac alternative. Verify before committing.

⚠ Accessibility permission

macOS will silently swallow clicks until you grant Terminal/Python under System Settings → Privacy → Accessibility. First run fails quietly otherwise.

💡 Always have a kill switch

Bind esc to a hard quit and keep cps modest while testing. A runaway clicker at 50 cps is hard to stop if it's clicking your own quit button.

💡 One flag, one writer

Only the hotkey handlers should write clicking. The loop only reads it. Keeps the two threads from stepping on each other.

WRAP-UP · WHAT THIS FEEDS

Where today goes next

Day 19

Screen recognition + window control. locateOnScreen() finds a button image, then your click code fires on it.

Days 20–21

Morning Launcher. The hotkey + decorator pattern triggers whole routines: tabs, Spotify, notifications.

CRUX tool-use

The thread + flag + global-trigger shape is exactly how CRUX will run background actions on command later.

You now have the core loop every later automation reuses — clicking is just the first action you bolt on.